

Week 3 Mini Project Assignment

Test Case File Pipeline — JSON, File Handling & REST APIs

Track	Python for QA Automation — Day 15 to Day 21
Project Type	Cumulative (builds on Week 1 + Week 2 code)
Target API	JSONPlaceholder (free, no auth) — https://jsonplaceholder.typicode.com/posts

Goal

Build a script that collects test cases from the user, saves them to a `.json` file, reads that file back, and pushes each test case to a free mock REST API. This simulates exactly what your future AI agent will do when pushing test cases to ADO/Jira.

Step by Step — What Your Script Must Do

STEP 1 — Collect test cases

Week 1 + 2 logic, reused

- Ask how many test cases the user wants to enter
- Wrap this in `try/except` using your `EnterWrongInput` custom exception — reject 0, negative numbers, or non-numeric input
- Loop that many times and collect for each test case: Title, Steps (comma separated → split into a list, stripped of extra spaces), Expected result, Actual result
- Call `create_test_case(title, steps, expected, actual)` to build the dict (function compares expected vs actual and auto-decides `passed` / `failed`)
- Append each returned dict into `test_case_list`
- **Print nothing until all inputs are collected**

STEP 2 — Write test cases to a .json file

Day 15 + 16

- Convert `test_case_list` to JSON using `json.dump()`
- Write it to a file named `test_cases.json` using `open()` in write mode
- Use `indent=4` so the file is human-readable

STEP 3 — Read the .json file back

Day 15 + 16

- Open `test_cases.json` in read mode
- Use `json.load()` to convert the file content back into a Python list of dicts
- Store this in a new variable (e.g. `loaded_test_cases`) — proves the round trip works

STEP 4 — POST each test case to JSONPlaceholder

Day 17 - 20

- Loop through `loaded_test_cases` using `enumerate()`
- For each test case, send a `POST` request using the `requests` library to `https://jsonplaceholder.typicode.com/posts`
- Send the test case dict as the JSON body
- Include headers: `{"Content-Type": "application/json"}`
- Wrap each request in `try/except requests.exceptions.RequestException` — if one call fails, print an error and continue (don't crash the script)

STEP 5 — Print a final report

- For each test case posted, print: test case number and title, status sent, API response status code (expect `201`), API-generated `id`
- After the loop, print a summary line with totals: total posted, success count, failed count

Code Structure Skeleton

```
import json
import requests

class EnterWrongInput(Exception):
    pass

def create_test_case(title, steps, expected, actual):
    # same logic as Week 2
    ...
    return { "title": title, "steps": steps, "expected": expected,
            "actual": actual, "status": status }
```

```
# ----- STEP 1: Collect inputs -----
test_case_list = []
try:
    count = int(input("How many test cases? = "))
    if count <= 0:
        raise EnterWrongInput("Please enter a number greater than 0")
except ValueError:
    print("Please enter only numbers")
    exit()
except EnterWrongInput as e:
    print(e)
    exit()

for i in range(1, count + 1):
    title = input(...)
    steps = [s.strip() for s in input(...).split(",")]
    expected = input(...).lower()
    actual = input(...).lower()
    test_case_list.append(create_test_case(title, steps, expected, actual))

# ----- STEP 2: Write to JSON file -----
with open("test_cases.json", "w") as f:
    json.dump(test_case_list, f, indent=4)

# ----- STEP 3: Read back from JSON file -----
with open("test_cases.json", "r") as f:
    loaded_test_cases = json.load(f)

# ----- STEP 4: POST each test case -----
url = "https://jsonplaceholder.typicode.com/posts"
headers = {"Content-Type": "application/json"}
success_count = 0
failed_count = 0

for index, tc in enumerate(loaded_test_cases, start=1):
    try:
        response = requests.post(url, json=tc, headers=headers)
        # check response.status_code, increment success_count
    except requests.exceptions.RequestException as e:
        # increment failed_count, print error

# ----- STEP 5: Final report -----
# print per-test-case results + summary totals
```

Expected Output (Sample)

```
=====
TEST CASE FILE PIPELINE - WEEK 3
=====

[STEP 1] Collecting test cases...
How many number of Test Cases you want to enter = 2
----- Test Case #1 -----
Enter Test Case#1 Title: Login with valid credentials
Enter Test Case#1 Steps (comma separated): Open browser, Enter URL, Click login
Enter Test Case#1 Expected Value: dashboard loaded
Enter Test Case#1 Actual Value: dashboard loaded
----- Test Case #2 -----
Enter Test Case#2 Title: Login with invalid password
Enter Test Case#2 Steps (comma separated): Open browser, Enter URL, Enter wrong password
Enter Test Case#2 Expected Value: error message shown
Enter Test Case#2 Actual Value: page crashed

[STEP 2] Writing test cases to test_cases.json ... Done
[STEP 3] Reading test cases back from test_cases.json ... Done

[STEP 4] Posting test cases to JSONPlaceholder API...

=====
POSTING REPORT
=====
Test Case 1: Login with valid credentials
  Status Sent      : passed
  API Status Code  : 201
  API Generated ID : 101
-----
Test Case 2: Login with invalid password
  Status Sent      : failed
  API Status Code  : 201
  API Generated ID : 102
-----

=====
Total Posted: 2 | Success: 2 | Failed: 0
=====
```

Concept	Where It's Used	Week
List of dicts, <code>enumerate()</code>	Storing & numbering test cases	Week 1
<code>def</code> , custom <code>Exception</code> , <code>try/except</code>	<code>create_test_case()</code> , input validation	Week 2
<code>json.dump()</code>	Writing test cases to file	Week 3
<code>json.load()</code>	Reading test cases back from file	Week 3
File handling (<code>open</code> , <code>with</code>)	Read/write <code>.json</code> file	Week 3
<code>requests.post()</code>	Sending test case data to API	Week 3
Headers (<code>Content-Type</code>)	Telling the API you're sending JSON	Week 3
<code>try/except</code> around API calls	One failed request shouldn't crash the script	Week 3
<code>response.status_code</code> , <code>response.json()</code>	Reading API response	Week 3

Acceptance Criteria Checklist

#	Criteria
1	Reuses <code>create_test_case()</code> and <code>EnterWrongInput</code> from Week 2
2	Test cases written to <code>test_cases.json</code> using <code>json.dump()</code> with <code>indent=4</code>
3	Test cases read back from the file using <code>json.load()</code> into a separate variable
4	Each test case from the loaded file is POSTed individually to JSONPlaceholder
5	Headers include <code>Content-Type: application/json</code>
6	Each API call wrapped in <code>try/except</code> — one failure doesn't crash the script
7	Final report shows status code + generated <code>id</code> per test case
8	Final summary line shows total posted / success / failed counts
9	Nothing printed during STEP 1 until all inputs are collected (Week 1 rule still applies)

- Notes before you start:**
- Install requests first: `pip install requests`
 - JSONPlaceholder is a fake API — it doesn't actually save your data. It always returns `201` with a made-up `id`. That's expected; the goal is practicing the request/response cycle, not real persistence.
 - Test each STEP in isolation (run STEP 1+2 first, check the file got created and looks right, then add STEP 3, then STEP 4) rather than writing the whole thing at once and debugging blind.